Question 1: Real-Time Face Recognition System (Edge Deployment)

1. Problem Understanding

The task is to deploy continuous face detection and identity matching, against a fixed known-person gallery, on a low-power single-board computer, while keeping latency low and remaining stable as ambient lighting changes throughout the day.

Identified Constraints:

- **A. Real-time throughput** — the pipeline must sustain a usable frame rate (target <30 FPS end-to-end latency) on a resource-constrained board.
- **B. Illumination robustness** — recognition accuracy must not collapse under low light, backlight, or uneven indoor lighting typical of edge deployment sites.
- **C. Resource ceiling** — Raspberry-Pi-class CPU (no dedicated GPU/NPU in the base case) and limited RAM (1–4 GB) rule out heavy deep networks run at full precision.
- **D. Closed-set recognition** — the system must match faces only against a small, predefined enrolled dataset, not perform open-world recognition.

2. Constraint Analysis & Prioritization

The four constraints interact rather than acting independently, so they must be resolved as a single optimisation rather than solved one at a time. Constraint C (limited CPU/RAM) is the binding constraint: it caps every downstream decision — model size, input resolution, and frame sampling rate. Constraint A (real-time latency) is therefore satisfied indirectly, by keeping the per-frame workload inside what constraint C allows, rather than by any single "speed" trick. Constraint B (lighting) is treated as a pre-processing concern that must be solved *before* detection/recognition, because a face detector trained or tuned on well-lit frontal faces degrades sharply on raw low-light frames — fixing illumination first prevents the more expensive stages downstream from wasting cycles on unusable input. Constraint D (closed-set matching) is actually a simplifying constraint: because the gallery is small and fixed, the recognizer can use a lightweight distance-based classifier (KNN / cosine similarity against pre-computed embeddings) instead of a large softmax classification head, which keeps memory and compute low. Priority order for design decisions: **C** → **A** → **B** → **D** — first fit the whole pipeline inside the CPU/RAM budget, then verify real-time behaviour under that budget, then harden against lighting, and finally optimise the closed-set matching step, which is the cheapest of the four once embeddings exist.

3. Proposed OpenCV Pipeline (Solution Development)

3.1 Image Acquisition

Use **cv2.VideoCapture** with the Pi camera (via the V4L2/GStreamer backend for lower overhead than a generic USB path) at a reduced capture resolution (e.g. 320×240 or 640×480 instead of 1080p). Frame skipping/sampling (process every 2nd–3rd frame, e.g. via a frame counter modulo N) is used to trade visual smoothness for detector throughput, since face identity does not change frame-to-frame. A separate capture thread (Python threading or OpenCV's own buffer) decouples camera I/O from processing so the pipeline is never blocked waiting on frame grabs.

3.2 Pre-processing

Each captured frame is converted to grayscale for the detection stage (cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)) to cut compute roughly in half versus 3-channel processing. Lighting robustness (constraint B) is handled with **CLAHE** (Contrast Limited Adaptive Histogram Equalization, cv2.createCLAHE) rather than plain global histogram equalization, because CLAHE normalises local contrast in shadowed and over-lit regions of the same frame without amplifying sensor noise. A light Gaussian blur (cv2.GaussianBlur, small 3×3 kernel) suppresses camera-sensor noise that CLAHE would otherwise amplify. Frames are resized to a fixed small working resolution before detection to bound worst-case latency.

3.3 Face Detection

A **Haar Cascade** (cv2.CascadeClassifier, haarcascade_frontalface_default.xml) or, where a small accuracy gain is worth a modest speed cost, OpenCV's **DNN face detector** (a quantised Caffe/TensorFlow SSD, run through cv2.dnn) is used instead of a heavy modern detector (MTCNN/RetinaFace), which are too expensive for a Pi CPU. Detection is run on the downscaled, CLAHE-corrected grayscale frame with detectMultiScale tuned (moderate scaleFactor ≈1.1–1.2, minNeighbors ≈4–5, a sensible minSize) to bound the number of pyramid levels searched, which is the dominant cost driver for Haar cascades.

3.4 Feature Extraction & Recognition

Detected face regions are cropped, resized to a fixed small patch (e.g. 96×96 or 112×112), and aligned using simple eye/landmark cues (cv2.face.createFacemarkLBF or a lightweight landmark model) so pose variation does not destabilise matching. Recognition uses a **pre-trained lightweight embedding model** (e.g. a MobileFaceNet-class network, exported to ONNX and executed through cv2.dnn, which avoids pulling in a full deep-learning framework runtime). Embeddings for the closed enrolled set (constraint D) are computed once offline and stored as a small vector table; live recognition reduces to a single nearest-neighbour cosine-similarity lookup against that table, which is O(N) over a small N and effectively free compared to detection.

3.5 Output / Visualization

Recognized identities are drawn with cv2.rectangle and cv2.putText directly onto the frame, and results are logged (timestamp, identity, confidence) to a lightweight local store (CSV/SQLite) so downstream consumers do not need the video stream itself.

4. Application of Concepts / Tools

This design applies: video I/O and threaded capture (cv2.VideoCapture), colour-space conversion and adaptive histogram equalization for illumination normalisation (CLAHE), classical cascade classifiers as a compute-cheap detector, the OpenCV DNN module for running a quantised deep embedding network without a full training framework, and nearest-neighbour/cosine similarity matching for closed-set identification. Model quantisation (INT8 where the exported ONNX model supports it) is applied to the embedding network specifically to fit constraint C.

5. Justification

A (real-time): grayscale conversion, reduced input resolution, frame skipping, and a cascade/quantised-SSD detector keep per-frame cost low enough that the pipeline sustains interactive frame rates on Pi-class hardware; the expensive embedding step only runs on the small cropped face region, not the full frame. **B (lighting):** CLAHE plus denoising specifically targets the failure mode of raw Pi-camera footage in non-ideal light, and is applied before detection so downstream stages see normalised input. **C (resources):** every component was chosen for its small memory/CPU footprint — Haar cascades and MobileFaceNet-class embeddings are explicitly designed for constrained hardware, and cv2.dnn avoids the RAM overhead of a full PyTorch/TensorFlow runtime. **D (closed set):** using stored embeddings with nearest-neighbour matching instead of a trained classifier means adding or removing an enrolled person requires no retraining — only updating the small embedding table — which also keeps the system's memory footprint proportional to the (small) gallery size.

Question 4: Facial Expression Recognition for Mobile Application

1. Problem Understanding

The task is to classify a user's facial expression (happy / sad / neutral) live through a phone camera, in a way that feels instantaneous and does not misfire every time the user moves slightly or the room lighting changes.

Identified Constraints:

- **A. Limited processing power** — must run on a mobile CPU (and ideally not drain battery or heat the device), so heavy CNN inference at full resolution is not viable per-frame.
- **B. Real-time performance** — expression feedback should feel immediate to the user, implying tight per-frame latency budgets.
- **C. Robustness** — small head pose changes and lighting shifts (typical of handheld phone use) must not cause expression flicker or misclassification.

2. Constraint Analysis & Prioritization

Constraint A again dominates: it forces a two-stage "detect cheap, classify small" architecture rather than running a single heavy end-to-end network on full camera frames. Constraint B is largely satisfied as a consequence of solving A correctly (small, well-scoped inputs are inherently fast), but also requires an explicit temporal smoothing strategy, because per-frame classification alone can flicker between adjacent expressions even when accuracy is high enough on average. Constraint C is handled specifically at the alignment step: without face alignment, small pose changes shift facial landmarks enough to confuse a lightweight classifier that was not trained with heavy augmentation. Practical priority: **A** $\rightarrow$ **C** $\rightarrow$ **B** — first shrink the problem to something a mobile CPU can run, then make that small pipeline pose/lighting-tolerant, and only then add the lightweight temporal smoothing that makes the already-fast pipeline feel real-time and stable to the user.

3. Proposed OpenCV Pipeline (Solution Development)

3.1 Face Detection

A Haar cascade or OpenCV DNN face detector (as in Q1, but tuned for a single dominant face — the phone's own user — rather than multiple faces) locates the face region each frame. Because the subject is close to and roughly centred on the camera, a restricted search region (region-of-interest cropped from the previous frame's detection, re-detected fully only every few frames) further cuts cost.

3.2 Pre-processing & Alignment

The cropped face is resized to a small fixed size (e.g. 48×48, the standard size used by lightweight expression datasets such as FER2013), converted to grayscale, and contrast-normalised with CLAHE or simple histogram equalisation to counter lighting changes (constraint C). A lightweight landmark/eye detector aligns the face (rotating so the eye line is horizontal) so that small head tilts do not shift facial geometry relative to the trained model's expectations.

3.3 Feature Extraction & Expression Classification

Two viable options, chosen by device tier: (i) a very small CNN (2–4 conv layers) trained on FER-style data and exported to a mobile-friendly format, run through cv2.dnn; or (ii) classical features — **Local Binary Patterns (LBP)** or **HOG** descriptors over the aligned face — feeding a lightweight classifier (linear SVM / small MLP). The classical route is preferred when constraint A is the tightest limit, since LBP/HOG extraction is extremely cheap and well supported natively in OpenCV, at a moderate accuracy cost versus a CNN.

3.4 Temporal Smoothing

Rather than trusting a single frame's prediction, a short rolling majority vote / exponential moving average over the last N (e.g. 5–8) predictions is used to decide the displayed expression. This directly targets constraint C's flicker risk from momentary pose blur or a single bad frame, at negligible extra cost.

3.5 Display

The recognised expression label (and optionally a confidence bar or emoji overlay) is rendered on-screen with cv2.putText and simple cv2.rectangle/overlay drawing, or fed to the app's native UI layer for a more polished presentation.

4. Application of Concepts / Tools

Applies cascade/DNN face detection, ROI-based tracking to avoid re-detecting every frame, CLAHE/histogram equalisation for illumination robustness, geometric alignment for pose robustness, classical texture descriptors (LBP/HOG) as a compute-cheap alternative to CNN features, lightweight classifiers (SVM/small MLP or a tiny CNN via cv2.dnn), and temporal smoothing (majority vote / moving average) for output stability.

5. Justification

A (limited CPU): small fixed input size, ROI-restricted re-detection, and classical LBP/HOG features (or a very shallow CNN) keep per-frame cost within mobile-CPU budgets without needing GPU acceleration. **B (real-time):** because every stage operates on a small cropped region rather than the full frame, and detection is only re-run in full periodically, the pipeline naturally sustains interactive frame rates; smoothing adds negligible overhead. **C (robustness):** CLAHE addresses lighting variance directly, alignment addresses small pose changes directly, and temporal smoothing absorbs the residual noise from both — together these keep classification stable without requiring a larger, slower, more "invariant" model.

Question 5: Smart Traffic Monitoring System

1. Problem Understanding

The task is to continuously monitor a live traffic camera feed to detect vehicles, follow each vehicle across frames, count crossings, and present live statistics to a city authority, all without falling behind the live feed.

Identified Constraints:

- **A. Vehicle detection** — must reliably find vehicles of varying size/type/distance in a live video stream from a fixed traffic camera.
- **B. Cross-frame tracking** — the same vehicle must be tracked as it moves across frames, not re-detected as a "new" object each frame.
- **C. Counting & statistics** — vehicles must be counted (e.g. crossing a line/zone) and summarised (counts per direction/time window).
- **D. Continuous, low-delay operation** — the system must run indefinitely (24/7) with minimal processing delay, implying stability as well as speed.

2. Constraint Analysis & Prioritization

This system has a natural pipeline dependency: constraint A (detection) must exist before B (tracking) can associate detections across frames, and B must exist before C (counting) can attribute a single crossing event to a single vehicle rather than double- or under-counting. Constraint D (continuous, low-delay operation) is a cross-cutting constraint that shapes *how* A–C are implemented rather than adding a separate stage: it pushes toward background-subtraction-based or lightweight-detector-based detection (cheap enough to run continuously for hours) and toward a lightweight tracker (e.g. centroid tracking or correlation trackers) rather than a heavy multi-object tracker, and it requires basic operational robustness (auto-recovery from dropped frames, periodic background-model refresh so lighting drift over a day does not degrade detection). Priority: **A** → **B** → **C** functionally, with **D** constraining the implementation choice at every one of those stages rather than being solved separately.

3. Proposed OpenCV Pipeline (Solution Development)

3.1 Video Handling

cv2.VideoCapture reads from the fixed traffic camera (RTSP stream or local feed). Since the camera is static, a background model can be built once and refreshed periodically. Frames are resized to a moderate working resolution to bound per-frame cost, and a counting line/zone (a fixed polyline in frame coordinates) is configured once for the camera's field of view.

3.2 Vehicle Detection

Two complementary approaches, chosen by required accuracy: (i) **Background subtraction** (cv2.createBackgroundSubtractorMOG2) for a fast, classical detector that isolates moving foreground blobs (vehicles) against the static road background — cheap enough for continuous 24/7 operation (constraint D); followed by contour extraction (cv2.findContours) and size/aspect-ratio filtering to reject noise (e.g. pedestrians, shadows, leaves). (ii) For higher accuracy in cluttered scenes, a lightweight pretrained object detector (e.g. a MobileNet-SSD or YOLO-tiny model run through cv2.dnn), restricted to vehicle classes, replaces/augments background subtraction. Morphological operations (cv2.morphologyEx with opening/closing) clean the foreground mask before contour extraction in the background-subtraction path.

3.3 Tracking

Each detected vehicle is assigned a persistent ID using a lightweight **centroid tracker**: detections in the new frame are matched to existing tracks by nearest centroid distance (optionally with a simple Kalman filter per track for motion prediction, smoothing out momentary missed detections). This is preferred over heavier deep-learning trackers/re-ID models specifically because of constraint D — centroid/Kalman tracking is cheap enough to run continuously alongside detection without becoming the bottleneck.

3.4 Counting & Statistics

A vehicle is counted the moment its tracked centroid crosses the pre-defined counting line, with a per-track "already counted" flag to prevent double-counting as the same vehicle continues to be tracked past the line. Counts are aggregated per direction (based on crossing trajectory) and per time window (e.g. per minute/hour) into a simple running-statistics structure (dictionary/CSV/lightweight DB) for later reporting.

3.5 Visualization

cv2.rectangle draws bounding boxes per tracked vehicle, cv2.putText labels each with its track ID, cv2.line draws the counting line, and a running count overlay (cv2.putText/simple on-frame panel) shows live statistics. Optionally, cv2.polylines draws recent trajectory trails per vehicle for operator diagnostics.

4. Application of Concepts / Tools

Applies background subtraction (MOG2) and morphological filtering for motion-based detection, contour analysis for blob extraction, an alternative lightweight DNN detector path via cv2.dnn for harder scenes, centroid/Kalman-filter based multi-object tracking for identity persistence, geometry-based line-crossing logic for event counting, and OpenCV drawing primitives for real-time operator visualization.

5. Justification

A (detection): background subtraction reliably isolates moving vehicles against a mostly static road scene at low cost, with an optional DNN path for scenes where classical methods are insufficient (occlusion, cluttered background). **B (tracking):** centroid/Kalman tracking gives each vehicle a stable ID across frames even through brief occlusion or a missed detection, which is essential before any counting logic can be trusted. **C (counting):** tying the count event to a track-level "crossed" flag (rather than counting raw per-frame detections) prevents the classic failure mode of counting one vehicle multiple times as it lingers near the line. **D (continuous, low-delay):** every chosen technique — MOG2, morphological ops, centroid tracking — is computationally lightweight and numerically stable over long runtimes, and periodic background-model refresh keeps detection accurate as lighting drifts across a full day/night cycle without manual intervention.

Question 6: Real-Time Object Detection on Mobile Device

1. Problem Understanding

The task is to detect general objects live through a phone's camera, in a way that feels instantaneous to the user and does not exhaust the phone's CPU, memory, or battery budget.

Identified Constraints:

- **A. Camera-based detection** — objects must be detected directly from the live mobile camera feed, in varied everyday scenes.
- **B. Real-time operation** — detection and display must keep pace with the live camera preview without perceptible lag.
- **C. Limited CPU/memory** — the app must run on typical phone hardware without excessive battery drain, heat, or memory pressure.

2. Constraint Analysis & Prioritization

Constraint C is again the binding constraint, and it directly shapes how A and B are met rather than being addressed separately. A general-purpose, high-accuracy detector (e.g. full YOLOv8 or Faster R-CNN at high resolution) would satisfy A well but would violate both B and C on typical mobile hardware, so the core design decision is choosing a detector family engineered specifically for mobile inference (SSD-MobileNet / YOLO-tiny / YOLO-nano class models) rather than trying to "optimise" a large model after the fact. Given that choice, B follows largely automatically, but is further protected with input-size reduction and frame-rate throttling so the UI thread is never blocked waiting on inference. Priority: **C** → **A** → **B** — pick a detector architecture that fits the device budget first; confirm it still detects the needed object classes adequately; then tune throughput/display so the experience feels real-time.

3. Proposed OpenCV Pipeline (Solution Development)

3.1 Image Acquisition

The mobile camera feed is captured via the platform camera API and handed to OpenCV as frames (cv2.VideoCapture in a Python/desktop prototype, or the equivalent native camera buffer on-device for OpenCV for Android/iOS / OpenCV.js in a hybrid app). Frames are captured at a modest preview resolution (e.g. 640×480) rather than full sensor resolution, since detection accuracy gains beyond that are marginal relative to the added cost.

3.2 Pre-processing

Each frame destined for inference is resized to the detector's expected small input size (e.g. 300×300 for SSD-MobileNet, or 320×320 for YOLO-nano), and normalised (scaled/mean-subtracted) as required by the specific model. The full-resolution frame is kept separately for display so downscaling for inference does not degrade what the user sees.

3.3 Object Detection

A pretrained, mobile-optimised detector — **SSD-MobileNet** or **YOLO-nano/tiny** — quantised to INT8/FP16 where the export toolchain supports it, is run through cv2.dnn (or the platform's native NNAPI/Core ML/GPU delegate where available for further speedup outside pure OpenCV). Non-maximum suppression (cv2.dnn.NMSBoxes) removes duplicate/overlapping boxes, and a confidence threshold filters low-quality detections before display.

3.4 Frame-Rate Management

Inference is decoupled from the camera preview loop: the preview renders every captured frame, but inference runs on a throttled cadence (e.g. every 2nd–4th frame, or on a dedicated lower-priority thread), with the last known detections drawn as an overlay on intermediate frames. This keeps the UI perceptibly smooth (constraint B) even if raw inference cannot match the full camera frame rate on a given device.

3.5 Display

Bounding boxes, class labels, and confidence scores are drawn on the full-resolution preview frame with cv2.rectangle and cv2.putText (or via the native UI overlay layer for a smoother-feeling app), giving the user an immediate visual result.

4. Application of Concepts / Tools

Applies mobile-optimised deep detector architectures (SSD-MobileNet / YOLO-nano) run through OpenCV's DNN module, model quantisation for footprint reduction, non-maximum suppression for clean output, decoupled capture/inference threading for perceived real-time behaviour, and OpenCV drawing utilities for overlay visualization.

5. Justification

A (camera-based detection): a pretrained general-object detector run on live camera frames directly satisfies detection-from-camera without needing a custom model per deployment. **B (real-time):** decoupling preview rendering from a throttled inference cadence means the app never visibly stalls, even on frames where inference is still running, satisfying the perceived real-time requirement independent of raw device speed. **C (limited CPU/memory):** choosing an architecture built for mobile (SSD-MobileNet/YOLO-nano) plus quantisation keeps both the model's memory footprint and per-inference compute within typical phone budgets, which a heavier detector could not guarantee regardless of downstream optimisation.

Question 9: Document Scanner Application

1. Problem Understanding

The task is to turn a handheld phone photo of a physical document into a clean, flat, high-contrast "scanned" image suitable for reading, sharing, or printing.

Identified Constraints:

- **A. Capture** — the app must reliably capture a document image from a handheld phone camera under imperfect real-world conditions (angle, shadows).
- **B. Readability enhancement** — the captured image must be enhanced so on-screen or printed text is clear and legible.
- **C. Clean output** — the final output should resemble a flat, cropped, properly-oriented "scan" rather than a raw photo of a document.

2. Constraint Analysis & Prioritization

Unlike the previous questions, this pipeline is not primarily latency-constrained — a scan is typically a one-shot, near-instant operation, so the dominant design driver is *geometric and photometric correction quality* rather than raw speed. Constraint A (capture) is mostly an acquisition/guidance concern: the app should help the user get a usable photo (edge overlay guidance) rather than only processing whatever is captured. Constraint C (clean, flat output) depends on solving a geometric problem first — finding the document's four corners and perspective-correcting them into a rectangle — before any pixel-level enhancement is meaningful, since enhancing a skewed photo still looks like a skewed photo. Constraint B (readability) is therefore sequenced *after* C's geometric correction: contrast/threshold enhancement is applied to the already-flattened document, not to the raw photo, so shadows and perspective-distorted lighting don't get baked into the final "clean" look. Priority: **A** → **C** → **B** — get a usable photo, flatten it geometrically, then enhance it photometrically.

3. Proposed OpenCV Pipeline (Solution Development)

3.1 Image Capture

The phone camera captures a still frame of the document (via the platform camera API into OpenCV as a standard BGR image). Optionally, live edge-detection feedback (rendering the pipeline below on the camera preview in real time) guides the user to position the document fully in frame before the shutter is pressed, directly supporting constraint A.

3.2 Pre-processing for Edge Detection

The captured image is resized down for fast processing (keeping the scale factor to map results back to full resolution later), converted to grayscale (cv2.cvtColor), blurred slightly (cv2.GaussianBlur) to suppress texture/noise that would otherwise create spurious edges, and run through **Canny edge detection** (cv2.Canny) to produce a clean edge map of the document boundary against its background.

3.3 Document Boundary Detection

Contours are extracted from the edge map (cv2.findContours), sorted by area, and the largest contour is approximated to a polygon (cv2.approxPolyDP). The pipeline looks specifically for a four-point (quadrilateral) contour, which is assumed to be the document's outline; the four corner points are then ordered consistently (top-left, top-right, bottom-right, bottom-left) for the next step.

3.4 Perspective Correction ("Flattening")

Given the four ordered corners and a computed target rectangle size (derived from the corners' side lengths), cv2.getPerspectiveTransform computes the transform matrix and cv2.warpPerspective applies it, producing a flat, straight-on view of the document regardless of the original capture angle — this is the step that most directly satisfies constraint C.

3.5 Readability Enhancement

On the flattened image: convert to grayscale; apply **adaptive thresholding** (cv2.adaptiveThresholding, Gaussian-weighted) or CLAHE-based contrast enhancement to normalise uneven lighting/shadows across the page and sharpen text-to-background contrast; optionally apply a light sharpening kernel (cv2.filter2D) to crispen text edges; and denoise (cv2.fastNlMeansDenoising) if the source photo was noisy. For a "black-and-white scan" look the final output can be pure binary; for a "colour/greyscale scan" look, contrast enhancement is applied without full binarisation.

3.6 Output

The enhanced, flattened image is saved (JPEG/PNG for sharing, or assembled into a PDF via the pdf toolchain for multi-page scans), giving the user a clean document file rather than a raw photo.

4. Application of Concepts / Tools

Applies edge detection (Canny), contour analysis and polygon approximation for document-boundary localisation, perspective transformation (homography via getPerspectiveTransform/warpPerspective) for geometric flattening, and adaptive thresholding/CLAHE/denoising/sharpening for photometric enhancement — a sequence that mirrors classical document-scanning pipelines.

5. Justification

A (capture): optional live edge-overlay guidance, plus robustness in the edge/contour stage to moderate angle and lighting variation, means the app tolerates realistic handheld capture rather than requiring a studio-quality photo. **C (clean output):** explicitly detecting the document's quadrilateral and perspective-warping it into a rectangle is what actually produces a "scanned" look — this is the step generic photo filters skip, and it is the distinguishing feature of a scanner app versus a camera app. **B (readability):** applying adaptive thresholding/CLAHE *after* flattening, rather than on the raw photo, ensures contrast correction responds to the document's actual (now-rectified) lighting pattern, giving consistently legible text output even when the original photo had uneven shadows across the page.
